

Processamento Digital de Sinais

DSP na ESP32

Fundamentos, Filtros Digitais, FFT e Aplicações Práticas

Aula 09 | Sistemas Embarcados com ESP32

Agenda

- 01 Fundamentos de DSP
- 02 Aquisição de Sinais na ESP32
- 03 Representação e Pré-processamento
- 04 Filtros Digitais (FIR e IIR)
- 05 Filtros Práticos na ESP32
- 06 Transformada de Fourier e FFT
- 07 Código Prático e Otimização
- 08 Aplicações Reais e Limitações

01

Fundamentos de DSP

Conceitos essenciais de processamento digital de sinais

O que é Processamento Digital de Sinais?

Definição

- Manipulação matemática de sinais representados digitalmente
- Converte sinais do mundo real em sequências numéricas para processamento

Etapas fundamentais

- **Amostragem** → Discretização no tempo
- **Quantização** → Discretização em amplitude
- **Processamento** → Filtragem, análise, transformação

Na ESP32

- ADC de 12 bits, dual-core 240 MHz, suporte a float

Sinal Analógico vs Sinal Digital

Sinal Analógico

- Contínuo no tempo e amplitude
- Infinitos valores possíveis
- Suscetível a ruído
- Ex: onda sonora, temperatura

$x(t) \in \mathbb{R}, t \in (-\infty, +\infty)$

Sinal Digital

- Discreto no tempo e amplitude
- Valores finitos (2^n níveis)
- Reprodutível e processável
- Ex: ADC da ESP32 (12 bits)

$x[n] \in \{0, \dots, 4095\}, n \in \mathbb{Z}$

Amostragem e Teorema de Nyquist

Teorema de Nyquist-Shannon

- Para reconstruir um sinal, a taxa de amostragem deve ser:

$$f_s \geq 2 \times f_{\max}$$

Exemplo prático

- Sinal de áudio até 20 kHz $\rightarrow f_s \geq 40$ kHz (CD usa 44.1 kHz)
- ESP32: ADC pode amostrar até ~6 kHz (modo padrão) ou ~2 MHz (DMA)

Aliasing

- Se $f_s < 2 \times f_{\max} \rightarrow$ frequências se sobrepõem (aliasing)

Quantização e Resolução

Conceito

- Mapeamento de valores contínuos para níveis discretos
- Introduz erro de quantização (inevitável)

Resolução na ESP32

- ADC de **12 bits** → $2^{12} = 4096$ níveis
- Faixa: 0 a 3.3V → Resolução = $3.3V / 4096 \approx 0.806 \text{ mV/bit}$

Erro de quantização

- Erro máximo = $\pm 1/2 \text{ LSB} = \pm 0.403 \text{ mV}$
- SNR teórica = $6.02 \times N + 1.76 \text{ dB} = 74 \text{ dB}$ (12 bits)
- Na prática, ESP32 atinge ~9-10 bits efetivos (ENOB)

02

Aquisição de Sinais na ESP32

ADC, taxa de amostragem, sincronização e DMA

ADC da ESP32 — Revisão

Características do ADC

- 2 unidades ADC (ADC1: 8 canais, ADC2: 10 canais)
- Resolução: 12 bits (configurável 9-12 bits)
- Atenuação configurável: 0 dB, 2.5 dB, 6 dB, 11 dB

Taxa de Amostragem

- analogRead(): ~100 µs/leitura ≈ **~6 kSPS**
- Modo I2S + DMA: até **~2 MSPS**
- Modo contínuo via driver adc_continuous: controle preciso

Sincronização de amostras

- Usar timers de hardware para intervalo fixo entre amostras

DMA e Aquisição Contínua

O que é DMA?

- Direct Memory Access
- Transfere dados ADC → RAM sem CPU
- CPU livre para processamento
- Ideal para DSP em tempo real

Vantagens

- Taxa constante de amostragem
- Zero jitter de software

Configuração

```
adc_continuous_handle_t h;  
  
adc_continuous_config_t cfg;  
  
cfg.sample_freq_hz = 20000;  
  
cfg.conv_mode = SINGLE;
```

Fluxo DMA

ADC → Buffer DMA → RAM

→ CPU processa bloco

→ Resultado (filtro/FFT)

Domínio do Tempo vs Frequência

Domínio do Tempo

- Amplitude vs tempo
- Visão direta da forma de onda
- Ideal para: picos, transições
- Gráfico: $x[n]$ vs n

"Quando algo aconteceu?"

Domínio da Frequência

- Magnitude vs frequência
- Revela componentes ocultas
- Ideal para: ruído, harmônicas
- Gráfico: $|X(f)|$ vs f

"Quais frequências estão presentes?"

Ruído e Pré-processamento

Tipos de ruído

- **Ruído branco:** distribuição uniforme em todas as frequências
- **Ruído rosa (1/f):** mais energia em baixas frequências
- **Interferência (50/60 Hz):** rede elétrica

Remoção de offset DC

- Subtrair média: $\hat{x}[n] = x[n] - \mu$
- Centraliza sinal em torno de zero

Normalização

- Escalar para faixa $[-1, 1]$ ou $[0, 1]$
- ESP32: raw / 4095.0 (normaliza para $[0, 1]$)

03

Filtros Digitais

FIR, IIR, média móvel e filtro exponencial

Filtro FIR — Finite Impulse Response

Conceito

- Saída depende apenas das entradas (sem feedback)
- Resposta ao impulso finita (N coeficientes)

$$y[n] = \sum b_k \cdot x[n-k]$$

Características

- Sempre estável (sem pólos)
- Fase linear possível (coefs simétricos)
- Complexidade: **O(N)** por amostra (N multiplicações + N somas)
- Maior N = melhor seletividade, mais CPU

Filtro IIR — Infinite Impulse Response

Conceito

- Saída depende de entradas **e saídas anteriores** (feedback)
- Resposta ao impulso teoricamente infinita

$$y[n] = \sum b_k x[n-k] - \sum a_k y[n-k]$$

Características

- Muito mais eficiente que FIR (menos coeficientes)
- Pode ser **instável** (pólos fora do círculo unitário)
- Fase não-linear (distorção de fase)
- Ideal para ESP32: menos operações por amostra

FIR vs IIR — Comparação

FIR

- ✓ Sempre estável
- ✓ Fase linear
- ✓ Simples de projetar
- ✗ Muitos coeficientes
- ✗ Mais memória e CPU
- ✗ Maior latência

IIR

- ✓ Muito eficiente (poucos coefs)
- ✓ Baixa latência
- ✓ Ideal para embarcados
- ✗ Pode ser instável
- ✗ Fase não-linear
- ✗ Sensível a arredondamento

Filtro de Média Móvel

Conceito — O filtro FIR mais simples

- Média das últimas N amostras

$$y[n] = (1/N) \sum x[n-k], \quad k=0..N-1$$

Implementação ESP32

```
#define N 10

float buf[N]; int idx = 0;

float media_movel(float x) {
    buf[idx] = x;
    idx = (idx+1) % N;
    float s=0;
    for(int i=0;i<N;i++) s+=buf[i];
    return s / N;
}
```

Filtro Exponencial (EMA)

Conceito — O filtro IIR mais simples

- Atenuação exponencial de amostras passadas
- Apenas 1 multiplicação + 1 soma por amostra

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1]$$

Parâmetro α ($0 < \alpha \leq 1$)

- α baixo (0.05) → suavização forte, resposta lenta
- α alto (0.9) → pouca suavização, resposta rápida

Implementação ESP32

```
float alpha = 0.1;

float filtrado = 0;

float ema(float x) {
    filtrado = alpha*x + (1-alpha)*filtrado;
    return filtrado; }
```

Comparação: Média Móvel vs Exponencial

Média Móvel

- Tipo: FIR
- Memória: **N floats**
- CPU: N somas + 1 divisão
- Latência: N/2 amostras
- Suavização: uniforme
- Melhor para: sinais estacionários

Exponencial (EMA)

- Tipo: IIR
- Memória: **1 float**
- CPU: 1 mult + 1 soma
- Latência: mínima
- Suavização: exponencial
- Melhor para: tempo real, IoT

04

Transformada de Fourier e FFT

Decomposição em frequências e análise espectral

Transformada de Fourier — Conceito Intuitivo

Analogia

- Como um prisma separa luz branca em cores (frequências)
- Fourier decompõe sinal complexo em ondas senoidais simples

Matemática (DFT)

$$X[k] = \sum x[n] \cdot e^{-j2\pi kn/N}$$

- $X[k]$ = componente na frequência k
- $|X[k]|$ = magnitude (potência naquela frequência)
- Complexidade DFT: $O(N^2)$ → impraticável para N grande

Resolução em frequência

- $\Delta f = f_s/N \rightarrow$ mais amostras = melhor resolução

FFT — Fast Fourier Transform

Por que FFT?

- DFT direta: $O(N^2)$ → $N=1024$: 1.048.576 operações
- FFT (Cooley-Tukey): $O(N \log N)$ → $N=1024$: 10.240 operações
- Redução de **~100x** para 1024 pontos!

Requisitos da FFT

- N deve ser potência de 2 (64, 128, 256, 512, 1024...)
- Resultado simétrico: apenas $N/2$ bins são úteis
- Freq máxima detectável = $f_s/2$ (Nyquist)

Na ESP32

- FFT de 1024 pontos: ~2-5ms no ESP32 (240 MHz)
- Bibliotecas: arduinoFFT, ESP-DSP (otimizada para ESP32)

Interpretação do Espectro FFT

Eixo X (Bins)

- Cada bin = uma faixa de frequência
- $f_k = k \times (f_s / N)$
- Bins 0 a N/2 (metade útil)

Exemplo

- $N=1024, f_s=10\text{kHz}$
- $\Delta f = 10000/1024 \approx 9.77 \text{ Hz}$

Eixo Y (Magnitude)

- $|X[k]|$ = intensidade da componente
- Pico = frequência dominante
- Múltiplos picos = harmônicas

Leitura prática

- $\text{Bin}[0]$ = componente DC
- $\text{Bin}[N/2] = f_s/2$ (Nyquist)
- Ignorar $\text{bin}[0]$ na busca de pico

FFT na ESP32 — Código (arduinoFFT)

```
#include <arduinoFFT.h>
#define SAMPLES 1024
#define SAMPLING_FREQ 10000
double vReal[SAMPLES], vImag[SAMPLES];
ArduinoFFT<double> FFT(vReal, vImag,
    SAMPLES, SAMPLING_FREQ);
// 1. Coletar amostras
for(int i=0; i<SAMPLES; i++) {
    vReal[i] = analogRead(34);
    vImag[i] = 0;
    delayMicroseconds(100); }
// 2. Janela + FFT + Magnitude
FFT.windowing(FFTWindow::Hamming,
    FFTDirection::Forward);
FFT.compute(FFTDirection::Forward);
FFT.complexToMagnitude();
// 3. Frequência dominante
```

Identificação de Frequência Dominante

Algoritmo de busca de pico

```
double peakFreq = 0; double peakVal = 0;

for(int i=1; i<SAMPLES/2; i++) {

    if(vReal[i] > peakVal) {

        peakVal = vReal[i];

        peakFreq = i * (SAMPLING_FREQ/SAMPLES);

    }

}
```

Fluxo completo

- 1. Coletar N amostras do ADC
- 2. Aplicar janela (Hamming/Hanning)
- 3. Executar FFT → complexToMagnitude
- 4. Buscar pico no espectro (ignorar bin(0))
- 5. Converter bin → frequência em Hz

05

Otimização de DSP na ESP32

DMA, buffers circulares, fixed-point e bibliotecas otimizadas

Biblioteca ESP-DSP

Biblioteca oficial da Espressif

- Otimizada para hardware ESP32 (instruções Xtensa)
- Até **10x mais rápida** que implementações genéricas

Módulos principais

- **dsps_fft2r**: FFT radix-2 (float e fixed-point)
- **dsps_biquad**: filtros IIR biquad (low/high/band-pass)
- **dsps_fir**: filtros FIR otimizados
- **dsps_mul/add**: operações vetoriais

Como usar (PlatformIO)

```
lib_deps = espressif/esp-dsp@^1.3.0
```

```
#include "dsps_fft2r.h"
```

Buffers Circulares e Processamento em Blocos

Buffer Circular

- Estrutura FIFO com tamanho fixo
- Sem cópia de memória ($O(1)$)
- Índice modular: $\text{idx} \% N$
- Ideal para filtros FIR

```
buf[head++ % N] = sample;
```

```
oldest = buf[(head-N) % N];
```

Processamento em Blocos

- Processar N amostras de uma vez
- Melhor uso do cache da CPU
- Reduz overhead de chamadas
- Compatível com DMA

Double Buffering

- Buffer A: DMA preenche
- Buffer B: CPU processa
- Alterna ao completar bloco

Float vs Fixed-Point na ESP32

Float (ponto flutuante)

- ✓ ESP32 tem FPU nativa!
- ✓ Fácil de programar
- ✓ Alta precisão
- ✗ Consome mais energia
- ✗ 4 bytes por valor

Recomendado para maioria dos casos na ESP32

Fixed-Point (int16/int32)

- ✓ 2 bytes por valor (int16)
- ✓ Menor consumo de energia
- ✓ Mais rápido em MCUs sem FPU
- ✗ Código mais complexo
- ✗ Risco de overflow

Usar quando memória é crítica

Arquitetura e Pipeline de Processamento

Pipeline completo de DSP na ESP32

Sensor → ADC → DMA → Buffer → Pré-proc → Filtro → FFT → Resultado

Detalhamento dos estágios

1. **Aquisição:** ADC + DMA captura amostras continuamente
2. **Pré-processamento:** remoção DC + normalização
3. **Filtragem:** EMA para suavização ou biquad para banda
4. **Análise:** FFT + busca de pico + classificação
5. **Ação:** alarme, log, transmissão MQTT/BLE

06

Aplicações Reais

Áudio, vibração, monitoramento elétrico e detecção de falhas

Análise de Áudio e Detecção de Vibração

Análise de Áudio

- Microfone MEMS + ADC
- FFT para espectro de frequência
- Detecção de notas musicais
- Equalizador gráfico em tempo real
- Monitoramento de nível sonoro (dB)

$f_s = 44.1 \text{ kHz}$ | FFT: 1024 pts

Detecção de Vibração

- Acelerômetro (MPU6050/ADXL345)
- Filtro passa-alta remove gravidade
- FFT identifica frequência de vibração
- Manutenção preditiva de máquinas
- Detecção de anomalias estruturais

$f_s = 1\text{--}10 \text{ kHz}$ | FFT: 512 pts

Monitoramento Elétrico e Detecção de Falhas

Sinais Elétricos

- Sensor de corrente (SCT-013)
- Análise de qualidade de energia
- FFT detecta harmônicas da rede
- Medição de THD
- Potência ativa, reativa, aparente

Fundamental: 50/60 Hz

Detecção de Falhas

- Mudança no espectro = anomalia
- Baseline espectral + comparação
- Threshold em bandas de frequência
- Alerta automático via MQTT/WiFi

Casos de uso

- Rolamentos de motores
- Compressores industriais

Problemas e Limitações

Aliasing

- Se f_s insuficiente → frequências fantasma no espectro
- Solução: filtro anti-aliasing analógico antes do ADC

Limitações do ADC ESP32

- Não-linearidade intrínseca (~9-10 ENOB)
- Ruído do WiFi afeta ADC2
- Solução: calibração + oversampling + ADC externo (ADS1115)

Limitações de processamento

- RAM limitada: 520 KB (cuidado com buffers grandes)
- FFT > 4096 pontos: possível, mas consome muita memória
- Sem FPU de precisão dupla → preferir float (32 bits)

Resumo e Próximos Passos

DSP na ESP32: de sinais analógicos a insights digitais

Fundamentos → Filtros → FFT → Aplicações

Aula 09 — Processamento Digital de Sinais